KFN PHARMA — Node.js + PostgreSQL

Site internet + application de gestion · installation et mise en ligne

Site internet (slider 3D, services, activités, équipe, contact, recherche de produits) et application **Système des ventes et de gestion de stocks** (siège + agences), réécrits en **Node.js (Express)** avec une base **PostgreSQL**. Responsive : ordinateur, tablette et téléphone.

1. Organisation des menus

L'application s'affiche avec un **menu latéral vert** (rubriques par section : Siège, Stocks, Ventes, Caisse, Éditions, Configuration ; chaque rubrique s'ouvre d'un clic ; le bouton rond réduit le menu aux icônes) et une **barre du haut** (profil et agence, bouton *Guide*, cloche des notifications, initiales de l'utilisateur avec *Se déconnecter*). Sur téléphone, le menu s'ouvre avec le bouton ☰.

Connexion : l'écran d'authentification s'affiche à chaque fois qu'on clique sur *Espace Connexion* depuis le site (une session encore ouverte est alors fermée). La session se ferme aussi à la fermeture du navigateur, après 30 minutes sans activité (réglable avec `SESSION_IDLE_MINUTES`) et par *Se déconnecter* ; le bouton « Précédent » ne réaffiche pas l'application après une déconnexion.

- **Agence Siège** : familles, articles, enregistrement du stock, appro du dépôt principal, appro des agences, compte des agences, objectifs, histogramme.
- **Gestion de stocks** : pré-commande, réception des appros, stocks, inventaires, péremption.
- **Ventes articles** : factures / pro-forma, modification, clients, **prix de vente par agence**.
- **Paiement caisse** : paiements du jour, impayés, dépenses, fermeture, arrêts.
- **Éditions des états** : les 12 bilans, **point des appros par agence**, arrêts de caisse, **statistique de vente par agence**.
- **ADMIN** : utilisateurs, agences, **fournisseurs**, notifications WhatsApp, **messages du site web**, modification des fichiers, import / export.

2. Règles de gestion

- **Les clients de l'AGENCE SIEGE sont les autres agences** ; chaque autre agence a ses propres clients, qu'elle crée, facture et encaisse.
- **Le siège ne vend pas aux agences.** Les agences s'approvisionnent uniquement par les **pré-commandes** : pré-commande de l'agence → validation par le siège (quantités accordées) → acceptation par l'agence (une quantité refusée peut être retournée au siège).
- **Une pré-commande acceptée devient l'impayé de l'agence** : une facture d'approvisionnement du siège est émise automatiquement à son nom.
- **Ces impayés se règlent dans *Compte des agences (clientes du siège)*** — et non à la caisse du siège :

- **suivi** de toutes les agences : total facturé, versé, reste (solde dû), nombre de factures impayées, plus ancien impayé, dernier versement ;
- par agence : ses factures d'approvisionnement (soldée / partiellement réglée / impayée, détail des articles et des paiements) ;
- **versement** : montant, moyen, référence ; on coche les factures à régler, ou le montant est réparti automatiquement des plus anciennes aux plus récentes ; un **reçu de versement** est imprimable ;
- **détails des versements** : dû avant, versé, solde après, factures réglées, saisi par ; annulation possible (Super Admin, Administrateur, Responsable) ;
- **qui fait quoi** : le siège (caissier(ère), Administrateur, Super Admin) enregistre les versements ; l'agence consulte son compte dans *Éditions des états → Compte agence auprès du siège*.

3. Workflow des pré-commandes et notifications WhatsApp

1. **Lancement** — une agence enregistre une pré-commande : le siège la reçoit automatiquement dans l'application (*Agence Siège → Approvisionnement des agences*, compteur sur l'accueil) **et** reçoit un message WhatsApp avec le détail de la commande (n°, agence, articles, quantités, montants, total).
2. **Validation** — quand le siège valide, l'agence reçoit un message WhatsApp avec les quantités accordées article par article (signalées quand l'accordé est inférieur au demandé) et le montant accordé.
3. **Réception** — quand l'agence accepte les produits, et/ou en retourne au siège, le siège reçoit un message WhatsApp listant les produits acceptés, les produits refusés, la facture du siège émise et ce qu'il reste à traiter.

Notifications dans l'application (enveloppes) — en plus de WhatsApp, une enveloppe apparaît en haut de l'écran :

Enveloppe	Signification
Bleue	Nouvelle pré-commande reçue (siège) ou nouvelle réponse du siège (agence) : à traiter
Rouge (clignote)	En attente depuis trop longtemps (48 h par défaut, réglable par l'administrateur)
Verte	Traitée (validée par le siège / réceptionnée par l'agence)

Le chiffre sur l'enveloppe se met à jour toutes les minutes ; un clic ouvre la liste des notifications, chacune menant directement à l'écran de traitement.

Paramétrage : ADMIN → Notifications WhatsApp

- **Numéros** : numéros du siège (plusieurs possibles) et numéro(s) de chaque agence (aussi modifiables dans *Paramétrage des agences*). Les numéros ivoiriens à 10 chiffres sont acceptés tels quels (indicatif 225 ajouté automatiquement).
- **Service d'envoi** au choix :

Service	À savoir
WhatsApp Cloud API (Meta)	Officiel. Jeton permanent + identifiant du numéro. Pour écrire à tout moment, créer un modèle « Utility » (ex. `kfn_notification`, corps `KFN PHARMA : {{1}}`)
Twilio	Account SID, Auth Token, numéro WhatsApp Twilio
UltraMsg	Utilise un numéro WhatsApp existant (QR code) ; non officiel : prendre un numéro dédié
Webhook HTTP	Envoie `{to, telephone, message}` en JSON à l'adresse de votre choix (autre passerelle, n8n, Make…)
Aucun	Les messages sont préparés dans le journal avec un bouton « Ouvrir dans WhatsApp » pour les envoyer à la main

- **Messages à envoyer** : chaque étape peut être activée ou désactivée.
- **Test** : envoi d'un message de test à un numéro.
- **Journal** : historique de tous les messages (envoyé, en échec avec la raison, à envoyer manuellement), bouton « Renvoyer ».
- **Adresse de l'application** : si elle est renseignée, un lien direct vers l'écran à traiter est ajouté aux messages.

4. Modifier et supprimer — qui peut le faire ?

Seuls le **Super Admin**, l'**Administrateur** et le **Responsable d'agence** voient et peuvent utiliser les boutons **Modifier** / **Supprimer** (ou **Annuler**). Les autres profils peuvent saisir, mais pas corriger ni effacer :

Enregistrement	Modifier	Supprimer / annuler
Familles	Personnes du siège	Super Admin (si non utilisée)
Articles, fournisseurs, clients	Oui	Oui (si non utilisé)
Prix de vente par agence	Oui	—
Pré-commandes en attente du siège	Quantités	Oui
Factures et pro-forma	Oui	Oui (si aucun paiement)
Paiements de caisse	—	Annuler (caisse non fermée)
Dépenses	Oui	Oui (caisse non fermée)
Fermeture de caisse	—	Rouvrir la caisse
Approvisionnement du dépôt principal	—	Annuler : Super Admin uniquement (si le stock le permet)
Retraits pour péremption	—	Annuler (quantité remise en stock)

Le contrôle est aussi fait par le serveur : un autre profil reçoit un message de refus.

Règles particulières

- **Familles d'articles** : un clic sur une ligne place la famille dans le champ ; **Modifier** est réservé aux personnes configurées au siège, **Supprimer** au Super Admin (et seulement si la famille ne contient aucun article).
- **Approvisionnement du dépôt principal** : le bouton **Annuler** n'est visible que par le Super Admin.
- **Ventes et caisse** : seul un opérateur **configuré dans l'agence concernée** peut agir (facturer, encaisser, dépenses, fermeture de caisse). Une personne du siège, même avec tous les droits, peut ouvrir les écrans d'une autre agence en consultation, mais pas faire d'opération.

5. Fiches imprimables

- **Pré-commande** : fiche après chaque enregistrement.
- **Validation par le siège** (Approvisionnement des agences) : fiche de validation (demandé / accordé) après chaque enregistrement.
- **Inventaire** : fiche d'inventaire imprimable ou enregistrable en PDF après validation (et réédition des fiches précédentes).

6. Inventaire de stock

- **Date initiale** : date du dernier inventaire de l'agence, fixe ; pour un premier inventaire, date du jour, fixe. **Au** : date du jour, modifiable.
- Colonnes : **Stock dernier inventaire** (qté réelle du dernier inventaire) · **Stock appro sur période** (date initiale → au) · **Stock départ** (= dernier inventaire + appros) · **Stock actuel logiciel** · **Inventaire physique** (saisie) · **Écart** (= logiciel − physique) · **Qté réelle** (stock logiciel par défaut, modifiable : décision du responsable).
- À la validation, un message prévient que **la QTE RÉELLE remplacera le stock de l'agence** ; l'écart est historisé.

7. Visibilité du stock

Valeur du stock **au prix d'achat**, au prix de vente, et **bénéfice** (= valeur vente − valeur achat), au total et par article.

8. Détails dans les états

Dans *Éditions des états*, choisissez les critères (agence, dates ; pour les ventes du jour aussi le client, le produit et l'état de paiement) : une ligne = une commande. Touchez une ligne pour ouvrir son détail, selon la rubrique : - **une facture** (ventes du jour, crédits sur période, restes à payer, clients sur période, crédits de plus de 45 jours, paiements, compte client) : articles vendus, paiements reçus, et bouton Imprimer ; - **un client** (bilans crédits, chiffre d'affaires par client) : ses factures, puis le détail de chacune ; - **un produit** (bilans produits) : toutes les factures où il a été vendu ; - **un approvisionnement** (bilans approvisionnements) : les articles approvisionnés ; - **une pré-commande** (*Point des appros par agence* : une ligne = une pré-

commande, filtres agence, dates, produit, état) : agence, produits, quantités demandées et accordées, état de chaque produit, dates.

Statistique de vente par agence est dans *Éditions des états* : une personne du siège consulte toutes les agences, une personne d'agence uniquement la sienne.

9. Prérequis

- **Node.js 18 ou plus** (conseillé : 22 LTS) — https://nodejs.org
- **PostgreSQL 12 à 17** (local, ou base en ligne : Render, Neon, Supabase, Railway…)

10. Installation en local (Windows, macOS, Linux)

1. Installez Node.js et PostgreSQL. Avec pgAdmin ou `psql`, créez une base vide : `CREATE DATABASE kfnpharma;`
2. Décompressez le projet, ouvrez un terminal dans le dossier `kfnpharma-node`, puis tapez `npm install` et `copy .env.exemple .env` (macOS / Linux : `cp .env.exemple .env`).
3. Ouvrez `.env` et renseignez l'accès à PostgreSQL (`PGHOST`, `PGDATABASE`, `PGUSER`, `PGPASSWORD`) ou une `DATABASE_URL`.
4. Lancez : `npm start` → **les 37 tables et les données de départ sont créées automatiquement** au premier démarrage.
5. Ouvrez **http://localhost:3000** (site) et **http://localhost:3000/gestion** (application).

Comptes de départ (mots de passe chiffrés automatiquement à la 1re connexion — **changez-les aussitôt** dans *ADMIN → Utilisateurs*) :

- **superadmin / Super@2026** — Super Admin
- **admin / Admin@2026** — Administrateur

Commandes utiles : `npm run db:init` (crée les tables si absentes), `npm run db:reset` (**efface tout** et recrée), `npm run dev` (redémarrage automatique pendant le développement).

11. Mise en ligne

Important : InfinityFree n'exécute pas Node.js (seulement PHP). Pour cette version, choisissez un hébergeur compatible Node.js ; la version PHP/MySQL (KFNPHARMA_InfinityFree.zip) reste disponible pour InfinityFree.

Render (render.com) — le plus simple, avec offre gratuite

1. Mettez le dossier du projet sur GitHub (sans `node_modules` ni `.env`).
2. Render → **New → Blueprint** → choisissez le dépôt : le fichier `render.yaml` crée le service web **et** la base PostgreSQL.
3. Au premier démarrage, les tables sont créées automatiquement. Ouvrez l'adresse `https://…onrender.com`. Vérifiez les conditions actuelles de l'offre gratuite (mise en veille du service, durée de vie de la base gratuite) : pour une utilisation quotidienne par toutes les agences, prenez une offre payante.

Railway, Fly.io, Koyeb… : créez un service Node.js (commande `npm start`) et une base PostgreSQL, puis définissez `DATABASE_URL`, `SESSION_SECRET` et `SECURE_COOKIES=true` dans les variables d'environnement.

Base en ligne Neon / Supabase : copiez leur chaîne de connexion dans `DATABASE_URL` (elle contient `sslmode=require`).

Serveur VPS (Ubuntu) : `docker compose up -d` (fichiers `Dockerfile` et `docker-compose.yml` fournis), ou bien Node.js + PostgreSQL installés, `npm ci --omit=dev`, puis `pm2 start server.js --name kfnpharma` derrière Nginx avec HTTPS (Let's Encrypt).

12. Variables d'environnement (`.env`)

Variable	Rôle
`PORT`	Port HTTP (3000 par défaut)
`DATABASE_URL`	Connexion PostgreSQL complète (prioritaire)
`PGHOST`, `PGPORT`, `PGDATABASE`, `PGUSER`, `PGPASSWORD`	Connexion PostgreSQL détaillée
`PGSSL=true`	Base distante qui exige SSL
`SESSION_SECRET`	Phrase secrète des sessions — **à changer**
`SECURE_COOKIES=true`	À activer quand le site est en HTTPS
`SESSION_IDLE_MINUTES`	Déconnexion automatique après N minutes sans activité (30 par défaut, 5 minimum)

13. Arborescence

```
server.js                serveur Express (site, connexion, routeur de l'application)
src/config.js            lecture du .env
src/core.js              PostgreSQL, utilitaires, profils et droits, numérotation, stock
src/web.js               menus, droits par page, mise en page
src/site.js              site internet (accueil, contact, recherche)
src/ventes.js            factures, transfert siège → agence, facture d'appro
src/api.js · print.js    fenêtres de choix (JSON) · impressions
src/tables.js            métadonnées des tables (éditeur, import / export Excel)
src/pages/*.js           un fichier par écran de l'application
public/assets/           styles, scripts, images
database/schema.sql      structure PostgreSQL (37 tables)
database/seed.sql        données de départ
```

14. Profils

Profil	Code	Accès
Super Admin	SUP	Tout, toutes les agences
Administrateur	ADM	Tout, sauf modifier / supprimer dans *ADMIN → Modification des fichiers*
Responsable d'agence	RAG	Caisse + ventes + consultation + stock de son agence
Gestionnaire stock	GST	Uniquement le stock
Caissier(ère)	CAI	Caisse + consultation
Vendeur(se)	VEN	Ventes + consultation
Consultation	CON	États (au siège : aussi les statistiques de toutes les agences)

15. Fonctions

Toutes les fonctions de la version précédente sont reprises : familles, articles, stocks et prix par agence, approvisionnement du dépôt principal, circuit pré-commande → validation → réception avec retour au siège, point des appros par année, factures et pro-forma, modification, caisse (jour, impayés, dépenses, fermeture, arrêts), 12 états avec dates de début et de fin, **Statistique de vente par agence** (classement, graphiques), onglet **ADMIN** (utilisateurs, paramétrage des agences, modification des fichiers façon Excel, importation / exportation Excel avec modèles à remplir, import CSV), questions de confirmation (« Voulez-vous valider l'enregistrement ? », « Voulez-vous modifier cette opération ? », « Voulez-vous supprimer ? Attention ! … ») et messages « Enregistrement / Modification / Suppression réussi(e) ».

16. Base de données PostgreSQL

Les noms des tables et des colonnes de l'analyse WinDev sont conservés, **en minuscules** (PostgreSQL ne distingue pas la casse sans guillemets) : `T_Stock` → `t_stock`, `RefArticles` → `refarticles`… Les fichiers Excel de l'ancienne version peuvent être importés : les noms de colonnes sont reconnus sans tenir compte des majuscules. Sauvegarde : *ADMIN → Exportation des données (Excel)* et/ou `pg_dump kfnpharma > sauvegarde.sql`.